

Decisiones de Arquitectura: Prompts Configurables

Prompts de clasificación configurables sin recompilar.

1. Claves canónicas de prompts

Basado en el análisis del código actual en `GptClasificarDataProvider.cs` , se definen las siguientes claves:

Clave	Descripción	Ubicación actual (hardcoded)
<code>classification.phase1.system</code>	System message para clasificación TDN1	GptClasificarDataProvider, línea ~89-96
<code>classification.phase1.user</code>	User prompt template para clasificación TDN1	GptClasificarDataProvider, línea ~99-119
<code>classification.phase2.system</code>	System message para clasificación TDN2	GptClasificarDataProvider, línea ~192-198
<code>classification.phase2.user</code>	User prompt template para clasificación TDN2	GptClasificarDataProvider, línea ~200-218

Convenciones de naming:

- Formato: `{dominio}.{fase}.{role}`
- Dominio: `classification` (extensible a otros dominios futuros)
- Fase: `phase1` , `phase2`
- Role: `system` , `user`

2. Límites máximos de contenido

Restricciones técnicas:

Límite	Valor	Justificación
Longitud máxima de prompt	16,000 caracteres	Equivalente aprox. a 4,000 tokens (asumiendo 4 chars/token)
Longitud máxima total (system + user)	32,000 caracteres	Límite conservador para contexto GPT-4 (128k tokens disponibles)
Número de placeholders por prompt	Ilimitado	Validación por nombre de placeholder, no por cantidad

Validaciones en tiempo de guardado:

- Error si excede 16,000 caracteres por prompt individual
- Warning si excede 12,000 caracteres (soft limit)
- Validación de placeholders obligatorios presentes

3. Placeholders permitidos

Placeholders existentes en código actual:

Placeholder	Descripción	Usado en	Obligatorio
{contextoPrompt}	Instrucciones adicionales desde Instrucciones.Prompt	Phase1 User	No
{phase1Catalog}	Catálogo de familias TDN1 generado dinámicamente	Phase1 User	Sí
{phase2Catalog}	Catálogo de tipologías TDN2 por familia	Phase2 User	Sí
{tdn1Code}	Código de familia TDN1 resuelto en phase1	Phase2 User	Sí
{contextoTexto}	Contenido textual del documento (markdown/texto)	Phase1 User, Phase2 User	No
{resumenInstruction}	Instrucciones adicionales para generar resumen (si aplica)	Phase1 System/User, Phase2 System/User	No
{documentName}	Nombre del archivo del documento	Phase1 User (fallback)	No

Sintaxis de placeholders:

- Formato: {nombreCamelCase}
- Case-sensitive
- Sin espacios ni caracteres especiales
- Sustitución: reemplazar por cadena vacía si no está disponible (excepto obligatorios)

Validación de placeholders obligatorios por prompt:

- **Phase1 System:** ninguno obligatorio
- **Phase1 User:** {phase1Catalog} obligatorio
- **Phase2 System:** ninguno obligatorio
- **Phase2 User:** {tdn1Code} , {phase2Catalog} obligatorios

4. Decisiones de arquitectura

4.1 Versionado de prompts

Decisión: Versionado numérico secuencial (int) autoincrementado

Justificación:

- Simplicidad operativa (no requiere gestión manual de versiones)
- Orden cronológico implícito (mayor número = más reciente)
- Fácil comparación y rollback (Version - 1)
- Suficiente para auditoría básica

Alternativa descartada: Versionado semántico (string major.minor.patch)

- Pros: expresividad de cambios (breaking/feature/fix)
- Contras: complejidad innecesaria para prompts de texto, requiere parsing

Implementación:

- Campo Version tipo int NOT NULL
- Autoincremento por PromptKey (no global)
- Restricción única: (PromptKey , Version , Environment)

4.2 Scopes por entorno

Decisión: NO se utiliza campo Environment en la entidad

Justificación:

- Cada entorno (dev, pre, pro) tiene su propia base de datos separada

- No hay riesgo de cruce de datos entre entornos
- Simplifica el diseño: menos índices, menos restricciones, menos lógica de resolución
- Elimina la necesidad de filtrado por entorno en queries

Alternativa descartada: Campo `Environment` con valores `dev`, `pre`, `pro`, `null`

- Pros: todos los entornos en la misma tabla
- Contras: innecesario cuando BBDD están separadas, complica índices únicos y lógica de resolución

Implementación:

- Sin campo `Environment` en la entidad
- Restricción única simplificada: (`PromptKey`, `Version`)
- Lógica de resolución directa: buscar versión activa para (`PromptKey`, `IsActive = true`)

4.3 TTL de caché

Decisión: TTL de **120 segundos (2 minutos)** para caché en memoria

Justificación:

- Balance entre latencia y frescura de cambios
- Suficientemente corto para iteración rápida (cambio visible en 2 min máx)
- Suficientemente largo para evitar martilleo de BBDD (60 reads/hora en carga constante)

Configuración:

- TTL configurable via `appsettings.json` (`ClassificationPrompts:CacheTtlSeconds`)
- Default: 120 segundos
- Mínimo aceptable: 30 segundos
- Máximo recomendado: 300 segundos (5 min)

Invalidación proactiva:

- No forma parte del alcance; la frescura de cambios se garantiza mediante el TTL de caché.

4.4 Política de edición

Decisión: Inmutabilidad de prompts publicados + creación de nuevas versiones

Flujo operativo:

1. **Crear borrador** (API POST `/admin/prompts`) → `IsActive = false`
2. **Activar versión** (API PUT `/admin/prompts/{id}/activate`) → `IsActive = true`, desactivar versión anterior activa
3. **Rollback** (API POST `/admin/prompts/{id}/rollback`) → Activar versión anterior, desactivar actual

Reglas de negocio:

- Solo 1 versión activa por `PromptKey` en un momento dado
- No se permite edición del campo `Content` de una versión activa
- No se permite borrado físico (solo marcar como inactiva)
- Auditoría completa: `CreatedBy`, `UpdatedBy`, `PublishedBy`, `PublishedAtUtc`

Alternativa descartada: Edición in-place de versión activa

- Pros: simplicidad UI
- Contras: pérdida de trazabilidad, imposibilidad de rollback preciso

5. Modelo de datos

```

CREATE TABLE [dbo].[PromptTemplate] (
    [Id] BIGINT IDENTITY(1,1) PRIMARY KEY,
    [PromptKey] NVARCHAR(100) NOT NULL,
    [Version] INT NOT NULL,
    [Content] NVARCHAR(MAX) NOT NULL,
    [IsActive] BIT NOT NULL DEFAULT 0,
    [Description] NVARCHAR(500) NULL,
    [CreatedAtUtc] DATETIME2(7) NOT NULL DEFAULT GETUTCDATE(),
    [CreatedBy] NVARCHAR(100) NOT NULL,
    [UpdatedAtUtc] DATETIME2(7) NULL,
    [UpdatedBy] NVARCHAR(100) NULL,
    [PublishedAtUtc] DATETIME2(7) NULL,
    [PublishedBy] NVARCHAR(100) NULL,

    CONSTRAINT [UQ_PromptTemplate_Key_Version] UNIQUE ([PromptKey], [Version]),
    CONSTRAINT [CK_PromptTemplate_Content_Length] CHECK (LEN([Content]) <= 16000)
);

CREATE INDEX [IX_PromptTemplate_Key_Active]
ON [dbo].[PromptTemplate] ([PromptKey], [IsActive])
WHERE [IsActive] = 1;

```

Nota: No se incluye campo `Environment` porque cada entorno (dev/pre/pro) tiene su propia base de datos separada.

6. Configuración de fallback en appsettings.json

```

{
  "ClassificationPrompts": {
    "CacheTtlSeconds": 120,
    "Fallback": {
      "Phase1": {
        "System": "Eres un sistema experto en clasificación de documentos del sector inmobiliario español...",
        "User": "Prompt adicional de instrucciones (si aplica):\n{contextoPrompt}\n\nFamilias TDN1 disponibles:\n{phase1Catalog}\n\n..."
      },
      "Phase2": {
        "System": "Eres un sistema experto en clasificación de documentos del sector inmobiliario español...",
        "User": "Familia TDN1 resuelta: {tdn1Code}\n\nTipologías disponibles en esta familia:\n{phase2Catalog}\n\n..."
      }
    }
  }
}

```

7. Resolución de prompts en runtime

La resolución de prompts sigue la cadena: base de datos → caché en memoria → fallback en `appsettings.json`.

`GptClasificarDataProvider` consume `IClassificationPromptProvider`, que aplica esta lógica de resolución y respeta el TTL de caché configurado.